

Practical 3:

Sorting Algorithms: Comparison-Based, Counting-Based, and Digit-Based Strategies

Practical Aim: To implement and compare multiple sorting algorithms and develop the ability to reason about which sorting strategy suits a given situation.

Problem 1

A teacher has a stack of student answer sheets with marks written on them and needs to arrange them in order before entering grades. She tries three different methods: in the first, she repeatedly compares adjacent sheets and swaps them if they are out of order; in the second, she finds the lowest-marked sheet each time and places it at the front; in the third, she picks each sheet one by one and inserts it into its correct position among the already-arranged sheets. Implement all three methods and for each one, print the sorted order of marks. Describe how each of the three methods works in your own words. If the sheets were already in order before the teacher started, which method would finish the fastest and why?

CODE:

```
package prac3;

import java.util.Scanner;

public class problem1 {

    static void bubbleSort(int[] a) {
        int n = a.length;
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (a[j] > a[j + 1]) {
                    int temp = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = temp;
                }
            }
        }
    }

    static void selectionSort(int[] a) {
```

```
int n = a.length;
for (int i = 0; i < n - 1; i++) {
    int min = i;
    for (int j = i + 1; j < n; j++) {
        if (a[j] < a[min]) {
            min = j;
        }
    }
    int temp = a[i];
    a[i] = a[min];
    a[min] = temp;
}

static void insertionSort(int[] a) {
    int n = a.length;
    for (int i = 1; i < n; i++) {
        int key = a[i];
        int j = i - 1;

        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }

        a[j + 1] = key;
    }
}

static void printArray(int[] a) {
    for (int x : a) {
        System.out.print(x + " ");
    }
    System.out.println();
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter number of students: ");
    int n = sc.nextInt();

    int[] a = new int[n];
    int[] b = new int[n];
    int[] c = new int[n];
}
```

```
System.out.println("Enter marks:");
for (int i = 0; i < n; i++) {
    a[i] = sc.nextInt();
    b[i] = a[i];
    c[i] = a[i];
}

bubbleSort(a);
selectionSort(b);
insertionSort(c);

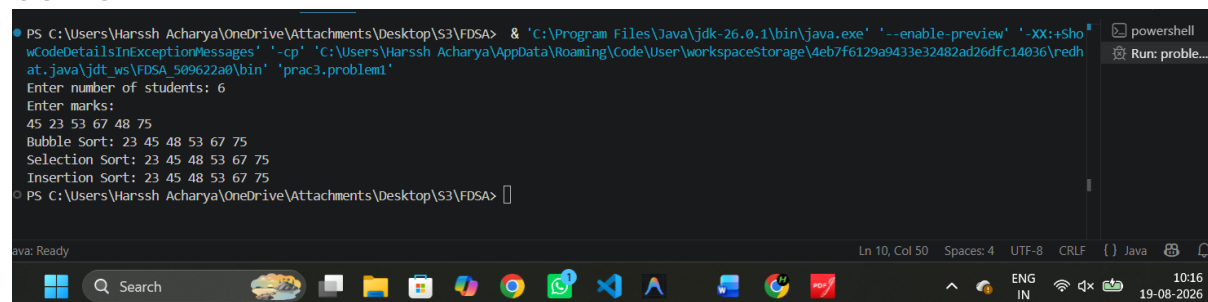
System.out.print("Bubble Sort: ");
printArray(a);

System.out.print("Selection Sort: ");
printArray(b);

System.out.print("Insertion Sort: ");
printArray(c);

sc.close();
}
}
```

OUTPUT:



```
PS C:\Users\Harssh Acharya\OneDrive\Attachments\Desktop\S3\FDSA> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+Sho
wCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Harssh Acharya\AppData\Roaming\Code\User\workspaceStorage\4eb7f6129a9433e32482ad26dfc14036\redh
at.java\jdt_ws\FDSA_509622a0\bin' 'prac3.problem1'
Enter number of students: 6
Enter marks:
45 23 53 67 48 75
Bubble Sort: 23 45 48 53 67 75
Selection Sort: 23 45 48 53 67 75
Insertion Sort: 23 45 48 53 67 75
PS C:\Users\Harssh Acharya\OneDrive\Attachments\Desktop\S3\FDSA>
```

Conclusion:

Bubble Sort, Selection Sort, and Insertion Sort successfully arrange the marks in ascending order. For an already sorted list, Insertion Sort is the fastest because it requires only one pass with minimal comparisons.

Problem 2

A paint shop has buckets labeled with one of three colour codes — 0, 1, or 2 — but they are stored in a random order. The shop owner wants all 0s together, then all 1s, then all 2s, without using any extra storage. Given the list of colour codes, rearrange them in place and print the result. Describe the approach you used. Does your solution make more than one pass through the list? Can you think of a way to do it in exactly one pass?

CODE:

```
package prac3;

import java.util.Scanner;

public class problem2 {

    static void sortColors(int[] a) {
        int low = 0;
        int mid = 0;
        int high = a.length - 1;

        while (mid <= high) {
            if (a[mid] == 0) {
                int temp = a[low];
                a[low] = a[mid];
                a[mid] = temp;
                low++;
                mid++;
            } else if (a[mid] == 1) {
                mid++;
            } else {
                int temp = a[mid];
                a[mid] = a[high];
                a[high] = temp;
                high--;
            }
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();

        int[] a = new int[n];
```

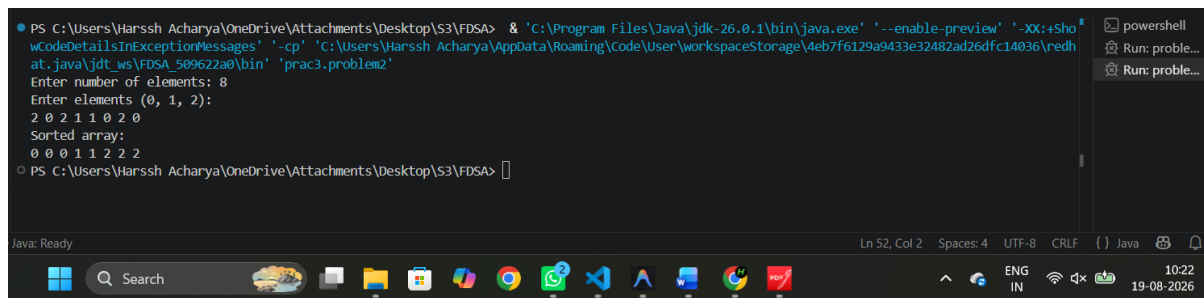
```
System.out.println("Enter elements (0, 1, 2):");
for (int i = 0; i < n; i++) {
    a[i] = sc.nextInt();
}

sortColors(a);

System.out.println("Sorted array:");
for (int x : a) {
    System.out.print(x + " ");
}

sc.close();
}
}
```

OUTPUT:



```
PS C:\Users\Harssh Acharya\OneDrive\Attachments\Desktop\S3\FDSA> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '-enable-preview' '-XX:+Sho
wCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Harssh Acharya\AppData\Roaming\Code\User\workspaceStorage\4eb7f6129a9433e32482ad26dfc14036\redh
at.java\jdt_ws\FDSA_509622a0\bin' 'prac3.problem2'
Enter number of elements: 8
Enter elements (0, 1, 2):
2 0 2 1 1 0 2 0
Sorted array:
0 0 0 1 1 2 2 2
PS C:\Users\Harssh Acharya\OneDrive\Attachments\Desktop\S3\FDSA>
```

Conclusion:

The array was sorted in-place using three pointers without extra storage. The Dutch National Flag approach sorts the array in a single pass with $O(n)$ time and $O(1)$ space.

Github link: <https://github.com/haarshaacharya/FDSA/tree/main/prac3>